

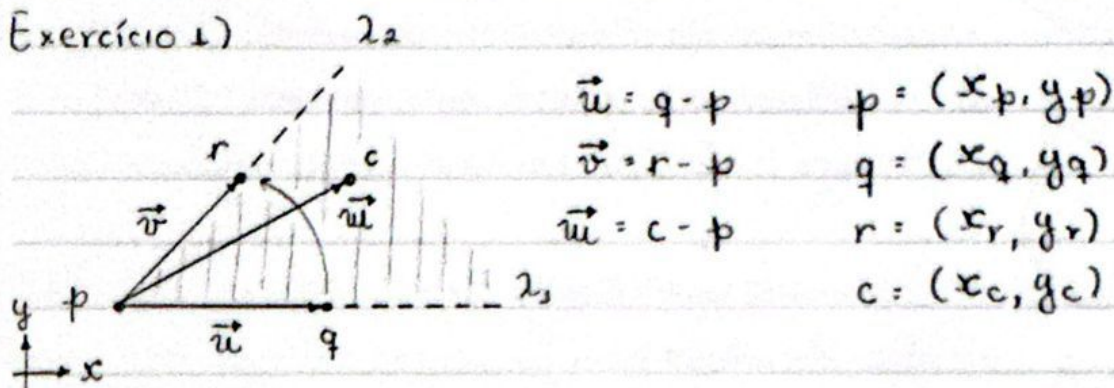
Disciplina : INF2604 - Geometria Computacional.

Professor : Waldemar Celes.

Aluno : Gabriel Ribeiro Gomes.

Avaliação conceitual.

Exercício 1)



Seja c um ponto qualquer no plano (exemplificado) na figura acima). Podemos representar um vetor $\vec{w}$ como sendo $\vec{w} = c - p$, da mesma forma como $\vec{v}$ e $\vec{u}$.

Podemos representar c como uma combinação linear de $\vec{v}$ e $\vec{u}$, a partir de p , usando λ_1 e λ_2 como as coordenadas baricêntricas: $c = p + \lambda_1 \vec{u} + \lambda_2 \vec{v}$

Para que c esteja na região delimitada por $\vec{u}$ e $\vec{v}$, precisamos garantir que $(\lambda_1, \lambda_2) > 0$. Podemos obter λ_1 e λ_2 a partir do cálculo do sistema linear:

$$\begin{aligned} \vec{w} &= c - p \\ c &= p + \lambda_1 \vec{u} + \lambda_2 \vec{v} \end{aligned} \quad \left\{ \begin{aligned} \vec{w} &= (p + \lambda_1 \vec{u} + \lambda_2 \vec{v}) - p \\ \vec{w} &= \lambda_1 \vec{u} + \lambda_2 \vec{v} \end{aligned} \right.$$

$$c - p = \lambda_1 (q - p) + \lambda_2 (r - p)$$

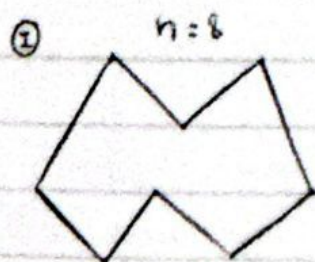
$$\begin{pmatrix} x_c - x_p \\ y_c - y_p \end{pmatrix} = \begin{pmatrix} \lambda_1 \\ \lambda_2 \end{pmatrix} \begin{pmatrix} x_q - x_p & x_r - x_p \\ y_q - y_p & y_r - y_p \end{pmatrix}$$

Assim, calculamos λ_1 e λ_2 , e testamos para as seguintes condições:

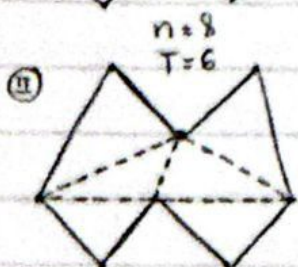
Se $\lambda_1 > 0$ e $\lambda_2 > 0$: o ponto c está na região.

Se não: o ponto c não está na região.

Exercício 2) Polígono qualquer com n vértices.

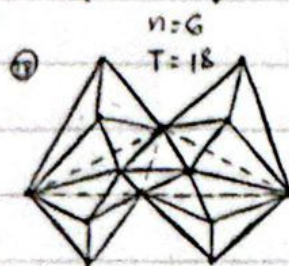
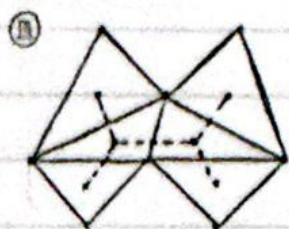


Sabemos que um polígono simples com n vértices tem $(n-2)$ triângulos. Colocando um ponto interno em cada triângulo, adicionamos $(n-2)$ vértices internos. Por fim, conectamos os pontos internos aos 3 vértices de seu triângulo correspondente.



Sabemos que $T = h + 2k - 2$ pelas propriedades da triangulação. Onde:

- h é o número de pontos do fecho convexo.
- k é o número de pontos interiores.



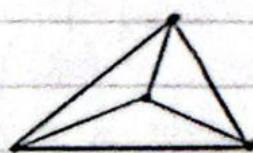
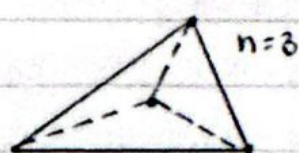
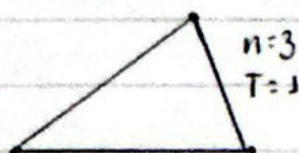
Nossa aresta externa aqui vale n , e o número de pontos internos vale $n-2$, portanto:

$$T = n + 2(n-2) - 2 = n + 2n - 4 - 2 = 3n - 6$$

$$\boxed{T = 3n - 6}$$

No exemplo acima, vemos que $n=8$, portanto $T = 3 \cdot 8 - 6 = 18$.

Podemos avaliar o caso mais simples de um triângulo:



$$n=3$$

$$T = 3 \cdot 3 - 6 = 9 - 6 = 3$$

Exercício 3) Fecho convexo F , pontos P , anti-horário, convexo.

$$F = \{p_0, p_1, p_2, \dots, p_{n-1}\}; p_i = (x_i, y_i)$$

$r = (x, y)$, adicionado a P , calcular o fecho.

Intuição geral: Precisamos inicialmente determinar se r está dentro ou fora de F . Se estiver dentro, o fecho não muda, se estiver fora, precisaremos encontrar os dois pontos de tangência em F a partir de r e remover os pontos.

1) Para os pontos em F (p_i de $i=0$ até $i=n-1$), calculamos a orientação O_i , em p_i, p_{i+1} e r ($O_i = \text{orientação}^*(p_i, p_{i+1}, r)$). Se todo O_i for maior ou igual a zero, então r já está no fecho convexo e não precisamos realizar o próximo passo. Já, no caso de encontrarmos $O_i < 0$, r estará à direita do segmento (p_i, p_{i+1}) , e precisaremos atualizar o fecho.

* a orientação se dá por:
$$\begin{bmatrix} x_{i+1} - x_i & y_{i+1} - y_i \\ x - x_i & y - y_i \end{bmatrix}$$

2) Para adicionar r ao fecho é necessário encontrar as tangentes superior e inferior. A tangente inferior é o ponto p_{inf} tal que o segmento (r, p_{inf}) é uma tangente inferior (no sentido horário) ao fecho. Analogamente, a tangente superior é p_{sup} tal que (r, p_{sup}) é tangente superior (anti-horário) ao fecho.

3) Após encontrar as tangentes retiramos os pontos do fecho que estão entre elas, conectando o ponto r as tangentes.

4) Por fim, atualizamos a lista com os segmentos do fecho resultante, mantendo a ordem anti-horária.

Assim, teremos o fecho convexo atualizado com complexidade $O(\log(n))$.

Exercício 4) Recebe uma semi-aresta da malha (he)

Retorna a próxima semi-aresta na borda (nb)

Inicialmente, avaliaremos se he está ou não na borda. Se he está na borda, o mate será nulo, então, percorreremos as semi-arestas conectadas ao vértice até encontrar a próxima borda. Já no caso de he não estar na borda, percorreremos as semi-arestas da borda diretamente, já que mate vai retornar $e.he_1$ ou $e.he_2$. Portanto, nosso algoritmo precisa apenas percorrer as semi-arestas até que he_i retorne mate nulo.

Pseudo-código que podemos utilizar:

```
getNextBorderHalfEdge( $he$ ):  
    edge =  $he$   
    while True:  
        edge = edge.he1  ← atualiza  
        ← na borda!  
        if getMate(edge) is null:  
            return edge ~  $nb$ 
```

Exercício 5) Ordem de complexidade das etapas do algoritmo de triangulação de Delaunay.

a) Ordenação de pontos.

→ Por mergesort poderemos ordenar em $O(n \log(n))$.

b) Construir um primeiro triângulo com 3 pontos iniciais.

→ Precisamos apenas acessar $P[0]$, $P[1]$ e $P[2]$, todos a $O(1)$, ou seja, tempo constante. Total: $O(1)$.

c) Construir as triangulações.

→ Precisamos verificar todos os n pontos, custando $O(n)$. Depois, construímos os novos triângulos considerando o ponto atual e as arestas da triangulação anterior, o que custa também $O(n)$. Fazendo isso para cada ponto, com n pontos a custo $O(n)$, temos um total de $O(n^2)$.

d) Operações de inversão até não ter arestas ilegais.

→ Realizamos no máximo $(n-2)(n-3)$ operações, que é o número de "flips" da operação, o que dá $O(n^2)$.

a) $O(n \log(n))$ → Custo total final: $O(n^2)$.

b) $O(1)$

c) $O(n^2)$

d) $O(n^2)$